```
#### Multiple Linear Regression ####

rm(list=ls())

# install required packages:
# install.packages("ggcorrplot")
# install.packages("ggplot2")
# install.packages("GGally")

#run the libraries
library(ggplot2)
library(ggcorrplot)
library(GGally)

##########################################################

#read in data with the read.csv file
fresh <- read.csv("D:\\UCT\\STA2020S 2025\\fresh.csv")

##########################################################
#slide 12
##########################################################

#correlation analysis
fresh_numeric <- fresh[,c(1,2,3,6)]
cor(fresh_numeric)

##########################################################
#slide 13
##########################################################

#correlation matrix plot
cor_matrix <- cor(fresh_numeric)
ggcorrplot(cor_matrix, lab = TRUE, type = "lower", colors =
c("red", "white", "blue"))

#pairwise plot
ggpairs(fresh_numeric)

##########################################################
#slide 19
##########################################################
```

```
fit <- lm(demand ~ fresh_price + ads_expenditure +
competitor_price, data = fresh)
summary(fit)

#########################################################
#slide 22
#########################################################

#test the significance of the beta1 estimate:
#manually calculate the test statistic
b2 <- 0.5012 #read off from summary(fit)
se_b2 <- 0.1259 #read from summary(fit)
tstat_b2 <- b2/se_b2 #calculates the test statistic
tstat_b2
#"manually" calculate the p-value
n <- 30
p <- 3
pval_b2 <- 2*pt(q=tstat_b2, df = n-(p+1), lower.tail = F)
#calculate the p-value for the test statistic (why do we say
2*pt() in this case?)
pval_b2
#OR just read it off from summary(fit)

#########################################################
#slide 25
#########################################################

#calculate confidence intervals for beta2 "manually"
#limits from the formulas:
ci_b2_upper <- b2 - qt(p=0.025, df = n-(p+1), lower.tail =
T)*se_b2
ci_b2_lower <- b2 + qt(p=0.025, df = n-(p+1), lower.tail =
T)*se_b2
print(paste("Beta2 CI: (",ci_b2_lower, ci_b2_upper,")"))
#OR with R's manual function:
confint(fit)

#########################################################
#slide 26 - 28
#########################################################

ybar <- mean(fresh$demand)
```

```
#"Manually" test for overall model significance (OR read it
off from the summary(fit) function)
#calculate tabled values
SSreg <- sum((fit$fitted.values-ybar)^2)
SSresid <-sum((fresh$demand-fit$fitted.values)^2)
SStot <- sum((fresh$demand-ybar)^2)
MSreg <- SSreg/p
MSresid <-SSresid/(n-(p+1))
#calculate the F statistic
Fteststat <-MSreg/MSresid
round(Fteststat, digits = 2)
#calculate the p-value
pval_Fteststat <- pf(q = Fteststat, df1 = p, df2 = n-(p+1),
lower.tail = F)
pval_Fteststat

##########################################################
#slide 29 - 31
##########################################################

#Manual calculation of R^2:
r2 <- SSreg/SStot
r2

#Manual calculation of adjusted R^2:
r2_adj <- 1- ((n-1)/(n-(p+1)))*(1-r2)
r2_adj

##########################################################
#slide 32
##########################################################

#assess the model assumptions

#check normality of residuals
#histogram of residuals
hist(fit$residuals)
#qqplot of residuals
qqnorm(fit$residuals)
qqline(fit$residuals, col = "red")

#assess assumption of constant variance
plot(fit$fitted.values, fit$residuals)
```

```
#assess assumption of independence of errors
plot(fresh$fresh_price, fit$residuals)
plot(fresh$ads_expenditure, fit$residuals)
plot(fresh$competitor_price, fit$residuals)

##########################################################
#slide 34
##########################################################

#prediction and confidence intervals with R's built-in
functions ONLY
#predict for a single individual (with its CI and PI)
#NB to take the units of measurement into account! R50 = 5,
R55 = 5.5, R8000 = 8, etc.
newdat <- list(fresh_price = 5, ads_expenditure = 8,
competitor_price = 5.5)
newdat <- data.frame(fresh_price = 5, ads_expenditure = 8,
competitor_price = 5.5)
predict(fit, newdata = newdat, interval = "confidence") #note
that it's narrower than the PI
predict(fit, newdata = newdat, interval = "prediction") #note
that it's wider than the CI

##########################################################
#slide 41
##########################################################

# Introducing a categorical variable with two levels

# with a default category (alphabetically)
fresh$size <- as.factor(fresh$size) #make it a factor
str(fresh) #check the type of your variables
contrasts(fresh$size) #shows how categorical variables are
represented as numerical values in a model

# with manual reference category
size_Big <- ifelse(fresh$size =="Small",0, 1) #if =small, then
assign 0, otherwise assign 1
size_Big <- factor(size_Big, labels = c("Small", "Big")) #make
it a factor
contrasts(size_Big)
fresh$size_Big <- size_Big #create new column in dataset
str(fresh)
```

```
# when calling str() you still see "1" and "2" assigned to the
size variable -
# this is fine; R knows what to do with it and you didn't do
anything wrong

#########################################################
#slide 43
#########################################################

#Including categorical variables in regression analysis

#default in R (alphabetically, so A will be the reference
category)
fresh$ads_campaign <- as.factor(fresh$ads_campaign)
str(fresh)

#manual reference category (suppose we want B to be the
reference category)
campA <- ifelse(fresh$ads_campaign=="A", 1, 0)
campC <- ifelse(fresh$ads_campaign=="C", 1, 0)
campA <- factor(campA, labels = c("not A", "A")) #make it a
factor with labels
campC <- factor(campC, labels = c("not C", "C"))

fresh$campA <- campA #add new columns to dataset
fresh$campC <- campC
str(fresh)


#########################################################
#slide 44
#########################################################

# fit the FULL MLR model
# Note that for fresh$ads_campaign, we will work with the
# default reference category A, and for fresh$size the
reference is 'Big'

fit <- lm(demand ~ fresh_price + ads_expenditure +
competitor_price
          + size + ads_campaign, data = fresh)
summary(fit)

#########################################################
```

```
#slide 51
#########################################################

#adding an interaction term to a regression model -
multiplicative model

fit_mult <- lm(demand ~ fresh_price*ads_campaign, data =
fresh)
summary(fit_mult)

#########################################################
#slide 54
#########################################################

# interaction plot for additive model with only fres_price and
# ads_campaign in MLR with ggplot
fit_ad <- lm(demand ~ fresh_price + ads_campaign, data =
fresh)
summary(fit_ad)
fresh$fitad <- predict(fit_ad)
# Plot
ggplot(fresh, aes(x = fresh_price, y = fitad, color =
ads_campaign)) +
  geom_line(size = 1) +
  labs(y = "Demand", x = "Fresh price") +
  xlim(3.55,3.9) +
  ylim(7,11) +
  theme_minimal()


# interaction plot for multiplicative model with fresh_price
and ads_campaign
# in MLR with ggplot
fresh$fitmult <- predict(fit_mult)
# Plot
ggplot(fresh, aes(x = fresh_price, y = fitmult, color =
ads_campaign)) +
  geom_line(size = 1) +
  labs(y = "Demand", x = "Fresh price") +
  xlim(3.55,3.9) +
  ylim(7,11) +
  theme_minimal()
```